

Learning Word Embeddings

Chapter 3 · From context to coordinates

Dr. A. Belcaid

Table of contents

1 From numbers to meaning **3**
1.1 One-hot vectors keep words separate . . . 3
1.2 Dense vectors can express graded relationships . . . 4

2 Similarity and semantic distance **4**
2.1 Euclidean distance . . . 4
2.2 Dot product . . . 4
2.3 Cosine similarity . . . 5
2.4 The classical analogy . . . 5

3 The distributional hypothesis **6**
3.1 What counts as context? . . . 6

4 Counting co-occurrence **7**
4.1 Constructing a matrix in Python . . . 8
4.2 Why raw counts are not enough . . . 8

5 Word2Vec: learn by predicting context **9**
5.1 Step 1: create positive pairs . . . 9
5.2 Step 2: store two vectors per word . . . 10
5.3 Step 3: score a pair . . . 10
5.4 Why not predict every vocabulary word? . . . 11

6 Negative sampling **11**
6.1 The learning loop . . . 12
6.2 A compact PyTorch model . . . 12
6.3 CBOW in one picture . . . 13

7 GloVe: learn from global counts **13**
7.1 Co-occurrence ratios carry information . . . 13
7.2 The GloVe objective . . . 13

7.3 Word2Vec and GloVe compared 14

8 Using pretrained embeddings 14
8.1 Well-known static embeddings 14
8.2 Query GloVe with Gensim 15
8.3 Use vectors through spaCy 15
8.4 Inspect neighborhoods, not isolated scores 16
8.5 Visualize a small selection carefully 16

9 What static embeddings cannot do 17
9.1 One word, one vector 17
9.2 Corpus boundaries become model boundaries 17
9.3 Frequency and vocabulary remain important 17
9.4 Bias is part of the geometry 17
9.5 Intrinsic success is not task success 18

10 Putting the ideas together 18

11 Check your understanding 18

12 Further reading and tools 19
12.1 Primary and explanatory readings 19
12.2 Practical tools 19
12.3 References 20

In Chapter 2, we used pretrained vectors as if their coordinates had simply appeared. Related words were nearby, cosine similarity measured their closeness, and vector arithmetic sometimes recovered familiar relationships. This chapter asks the missing question:

> How can a machine learn useful coordinates from ordinary text?

The central answer is surprisingly economical: **use context as supervision**. A corpus does not explicitly say that *coffee* and *tea* are related, but both occur near words such as *drink*, *cup*, *hot*, and *morning*. Embedding algorithms turn those repeated contextual patterns into geometry.

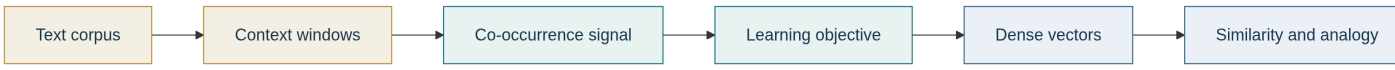


Figure 1: The chapter's central path: context supplies the training signal from which a useful geometry emerges.

Learning objectives

After this chapter, you should be able to:

1. distinguish token identifiers, one-hot vectors, and learned dense embeddings;
2. use distance, dot products, and cosine similarity appropriately;
3. explain the distributional hypothesis and construct a co-occurrence matrix;
4. turn a corpus into Skip-gram training pairs;
5. explain the essential ideas behind Word2Vec and negative sampling;
6. contrast Word2Vec's local prediction objective with GloVe's global count objective;
7. query pretrained Word2Vec, GloVe, fastText, and spaCy vectors; and
8. identify important limitations of static word embeddings.

1 From numbers to meaning

Suppose our vocabulary is:

```
0 → apple
1 → orange
2 → car
3 → bus
```

These integers are **identifiers**, not measurements. We must not conclude that *bus* is three times *orange*, or that *orange* is closer to *apple* than *bus* is. Changing the vocabulary order changes every identifier without changing the language.

1.1 One-hot vectors keep words separate

A one-hot vector gives every vocabulary item its own coordinate:

$$\begin{aligned} \text{apple} &\to [1,0,0,0], \\ \text{orange} &\to [0,1,0,0], \\ \text{car} &\to [0,0,1,0], \\ \text{bus} &\to [0,0,0,1]. \end{aligned}$$

This representation avoids pretending that vocabulary indices have numerical order. It also creates a new problem: all distinct words are equally dissimilar.

For any two different one-hot vectors $\mathbf{x}$ and $\mathbf{y}$,

$$\mathbf{x}^\top \mathbf{y} = 0 \qquad \text{and} \qquad \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{2}.$$

The representation knows that *apple* and *orange* are different, but not that both are fruit.

Table 1: Three ways of representing vocabulary items.

Representation	Size	Dense?	Encodes similarity?
Token ID	1	—	No
One-hot vector	Vocabulary size	No	No
Learned embedding	Usually 50–300	Yes	Often

1.2 Dense vectors can express graded relationships

An embedding assigns a short, dense vector to each word. For illustration, imagine a learned two-dimensional space:

```
apple  → [0.82, 0.16]
orange → [0.79, 0.22]
car    → [0.11, 0.85]
bus    → [0.16, 0.80]
```

The coordinate names are not usually known. What matters is their relationship: fruit vectors point in similar directions, as do vehicle vectors.

> **!** An embedding is useful because of the **geometry of the collection**, not because one coordinate has an obvious human-readable definition.

2 Similarity and semantic distance

Once words are vectors, we need a rule for comparing them.

2.1 Euclidean distance

Euclidean distance measures the straight-line separation between two points:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2.$$

Smaller values mean closer points. This is intuitive, but vector length can affect the result.

2.2 Dot product

The dot product is

$$\mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i.$$

It becomes large when two vectors align and have substantial magnitude. Word2Vec uses dot products as trainable compatibility scores.

2.3 Cosine similarity

Cosine similarity divides the dot product by both vector lengths:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^{\mathsf{T}}\mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \tag{1}$$

It asks whether the vectors point in similar directions. It is the standard first choice when querying static word embeddings.

```python
import numpy as np

def cosine_similarity(u, v):
    u = np.asarray(u, dtype=float)
    v = np.asarray(v, dtype=float)
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

apple = [0.82, 0.16]
orange = [0.79, 0.22]
car = [0.11, 0.85]

print(cosine_similarity(apple, orange))
print(cosine_similarity(apple, car))
```

> **⚠ Similarity is not identity**
>
> Nearest neighbors can be synonyms, but they can also be related concepts, grammatical alternatives, or opposites. *Hot* and *cold* occur in many of the same contexts and may therefore be close.

2.4 The classical analogy

The most famous word-vector demonstration is

$$\textbf{king} - \textbf{man} + \textbf{woman} \approx \textbf{queen}.$$

It suggests that some relations correspond approximately to directions in the space. We search for the word whose vector has the greatest cosine similarity to the result.

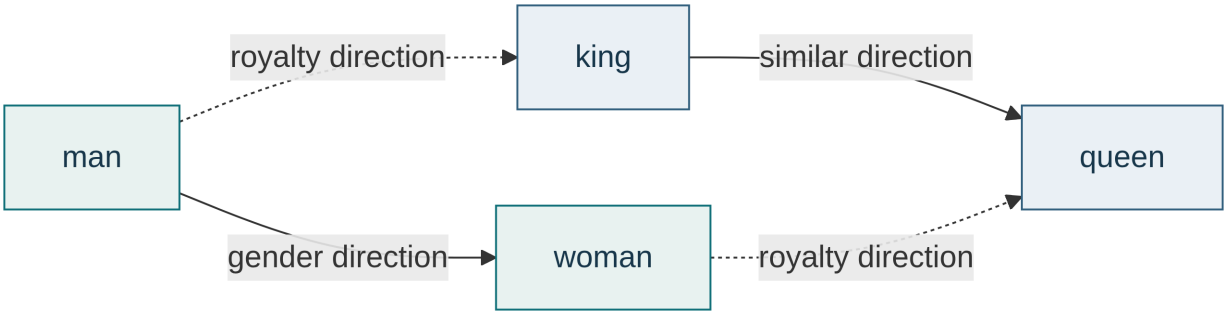


Figure 2: A word analogy asks whether two pairs exhibit approximately parallel displacements.

This is a memorable illustration, not a law of language. Analogy results depend on the corpus, algorithm, vocabulary, and search procedure.

3 The distributional hypothesis

Firth (1957) summarized a long-standing linguistic idea with the sentence "You shall know a word by the company it keeps." In computational terms:

> Words that occur in similar contexts tend to have related meanings.

Consider two unseen words:

```
She poured the dax into a cup.
The dax was hot and bitter.

She poured the blicket into a cup.
The blicket was hot and sweet.
```

Even without definitions, the shared contexts suggest that both words name drinks. The corpus provides **distributional evidence**.

3.1 What counts as context?

Context is a modeling decision. It might mean:

- neighboring words within a fixed window;
- all words in the same sentence or document;
- words connected by a grammatical dependency; or
- documents, users, or time periods associated with a word.

Classical Word2Vec normally uses a short window around each word. A narrow window often emphasizes functional similarity; a wider window can emphasize topical relatedness.

For the sentence

```
the curious student reads the book
```

with a window radius of two, the center word `student` has these context words:

```
the curious [student] reads the
 ↑     ↑                ↑    ↑
context                context
```

The ordered center–context pairs are:

```
(student, the)
(student, curious)
(student, reads)
(student, the)
```

Repeating this operation across a large corpus creates millions or billions of weak training examples without manual labels.

> **Corpus text supervises itself**
>
> No annotator labels two words as semantically related. Their observed neighborhoods supply the signal. This is an early and influential form of **self-supervised learning**.

4 Counting co-occurrence

The most direct distributional representation counts how frequently each word appears near each context word.

Consider the tiny corpus:

```
cats chase mice
dogs chase cats
dogs eat food
cats eat food
```

With a symmetric window of radius one, a simplified word–context matrix is:

Table 2: A small co-occurrence matrix. Rows represent center words and columns represent observed contexts.

Center word	cats	dogs	chase	eat	mice	food
cats	0	0	2	1	0	0
dogs	0	0	1	1	0	0
chase	2	1	0	0	1	0
eat	1	1	0	0	0	2

The rows for `cats` and `dogs` are similar because both appear beside `chase` and `eat`. Their distributional similarity becomes visible before we train a neural model.

4.1 Constructing a matrix in Python

```
from collections import Counter
import numpy as np

sentences = [
    "cats chase mice".split(),
    "dogs chase cats".split(),
    "dogs eat food".split(),
    "cats eat food".split(),
]

vocabulary = sorted({word for sentence in sentences for word in
 ↪  sentence})
word_to_id = {word: i for i, word in enumerate(vocabulary)}
counts = np.zeros((len(vocabulary), len(vocabulary)), dtype=int)

window = 1
for sentence in sentences:
    for center_index, center in enumerate(sentence):
        left = max(0, center_index - window)
        right = min(len(sentence), center_index + window + 1)
        for context_index in range(left, right):
            if context_index == center_index:
                continue
            context = sentence[context_index]
            counts[word_to_id[center], word_to_id[context]] += 1

print(vocabulary)
print(counts)
```

4.2 Why raw counts are not enough

Raw co-occurrence matrices have three major limitations:

1. **They are huge and sparse.** A vocabulary of 100,000 words produces 10^{10} possible cells.
2. **Frequency dominates.** Common function words occur with almost everything.
3. **Counts do not distinguish expected from surprising co-occurrence.** Ten observations may be informative for rare words but ordinary for frequent ones.

Pointwise mutual information compares observed co-occurrence with what independence would predict:

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}. \tag{2}$$

A positive value means that w and c occur together more often than chance would suggest. Positive PMI (PPMI) replaces negative values with zero.

> **Historical route: matrix reduction**
>
> Classical distributional methods can weight a word–context matrix and compress it with a technique such as singular value decomposition. This is an important historical route to dense representations, but it is not our focus. We now turn to methods that learn the vectors directly from a predictive or count-based objective.

5 Word2Vec: learn by predicting context

Word2Vec is a family of shallow neural models introduced by Mikolov, Chen, et al. (2013) and refined with efficient training strategies by Mikolov, Sutskever, et al. (2013). Its enduring contribution is not a complicated architecture. It is a clever way to transform local context prediction into useful word vectors.

Word2Vec has two principal variants:

Table 3: The two Word2Vec training directions.

Variant	Input	Prediction target
Skip-gram	Center word	Nearby context words
CBOW	Nearby context words	Center word

We focus on **Skip-gram with negative sampling**, because its mechanics expose the essential ideas most clearly.

5.1 Step 1: create positive pairs

For a token sequence $w_1, \ldots, w_T$ and window radius m, Skip-gram creates (w_t, w_{t+j}) pairs for $-m \le j \le m$, excluding $j = 0$ and positions outside the sentence.

```
def skipgram_pairs(tokens, window_size=2):
    pairs = []
    for center_index, center in enumerate(tokens):
        left = max(0, center_index - window_size)
        right = min(len(tokens), center_index + window_size + 1)

        for context_index in range(left, right):
            if context_index ≠ center_index:
                pairs.append((center, tokens[context_index]))
    return pairs
```

```
tokens = "the curious student reads the book".split()
print(skipgram_pairs(tokens, window_size=2))
```

Every observed pair receives the label 1: it is a **positive example**.

5.2 Step 2: store two vectors per word

Skip-gram maintains two parameter matrices:

$$W_{\text{center}} \in \mathbb{R}^{|V| \times d} \qquad \text{and} \qquad W_{\text{context}} \in \mathbb{R}^{|V| \times d}.$$

Selecting word w from the first matrix gives its center vector $\mathbf{v}_w$. Selecting context c from the second gives $\mathbf{u}_c$.

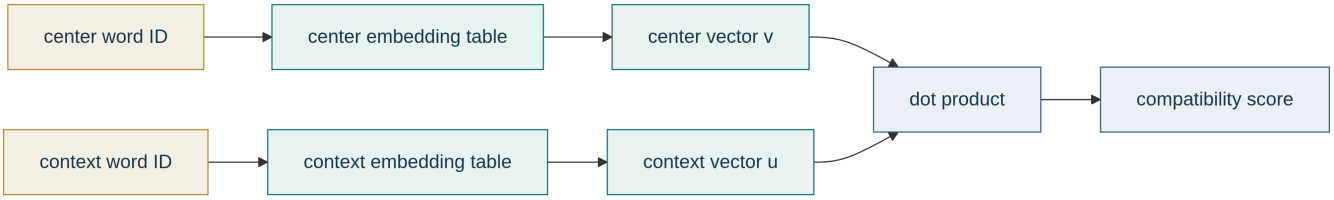


Figure 3: Skip-gram looks up a center vector and scores it against a context vector.

Conceptually, a one-hot word vector multiplied by an embedding matrix selects a row. Software performs this operation efficiently as a lookup rather than a full matrix multiplication.

5.3 Step 3: score a pair

The dot product

$$s(w, c) = \mathbf{u}_c^{\mathsf{T}} \mathbf{v}_w$$

scores compatibility. A sigmoid converts the score to a number between zero and one:

$$\sigma(s) = \frac{1}{1 + e^{-s}}.$$

For an observed pair, training should make $\sigma(\mathbf{u}_c^{\mathsf{T}} \mathbf{v}_w)$ large.

5.4 Why not predict every vocabulary word?

The original full prediction view assigns a probability to every possible context word:

$$P(c \mid w) = \frac{\exp(\mathbf{u}_c^\mathsf{T}\mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\mathsf{T}\mathbf{v}_w)}.$$

The denominator examines the entire vocabulary for every training pair. With millions of pairs and a large vocabulary, that is expensive.

6 Negative sampling

Negative sampling replaces the large multiclass problem with several small binary decisions:

> Is this center–context pair observed in the corpus, or was it artificially generated?

For the positive pair `(student, reads)`, sampled negative pairs might be:

```
(student, reads)      label 1
(student, pineapple)  label 0
(student, ocean)      label 0
(student, bicycle)    label 0
```

For one positive pair (w, c) and k sampled negative contexts $n_1, \dots, n_k$, we maximize

$$\log \sigma(\mathbf{u}_c^\mathsf{T}\mathbf{v}_w) + \sum_{i=1}^{k} \log \sigma(-\mathbf{u}_{n_i}^\mathsf{T}\mathbf{v}_w). \tag{3}$$

The first term rewards a high score for the observed pair. Each negative term rewards a low score for a noise pair.

> **The essential intuition**
>
> Training pulls observed center–context vectors into compatible directions and pushes sampled noise pairs apart. Words that repeatedly interact with similar contexts receive similar updates and gradually occupy nearby regions.

Negatives are commonly sampled from a smoothed unigram distribution rather than uniformly. The original implementation used word frequency raised to the $3/4$ power, balancing very frequent and very rare words (Mikolov, Sutskever, et al. 2013).

6.1 The learning loop



Figure 4: The Skip-gram learning loop repeats simple local decisions across a large corpus.

6.2 A compact PyTorch model

This model shows the central computation without implementing a complete training pipeline:

```
import torch
from torch import nn

class SkipGramNegativeSampling(nn.Module):
    def __init__(self, vocabulary_size, embedding_dim):
        super().__init__()
        self.center_embeddings = nn.Embedding(vocabulary_size,
        ↪  embedding_dim)
        self.context_embeddings = nn.Embedding(vocabulary_size,
        ↪  embedding_dim)

    def forward(self, center_ids, context_ids):
        center_vectors = self.center_embeddings(center_ids)
        context_vectors = self.context_embeddings(context_ids)
        return (center_vectors * context_vectors).sum(dim=-1)

model = SkipGramNegativeSampling(vocabulary_size=10_000,
↪  embedding_dim=100)

# Each position represents one positive or sampled-negative pair.
center_ids = torch.tensor([12, 12, 12, 12])
context_ids = torch.tensor([98, 401, 57, 806])
labels = torch.tensor([1.0, 0.0, 0.0, 0.0])

scores = model(center_ids, context_ids)
loss = nn.functional.binary_cross_entropy_with_logits(scores, labels)
loss.backward()
```

`binary_cross_entropy_with_logits` combines the sigmoid and binary loss in a numerically stable operation.

> **What happens to the two embedding tables?**
>
> Word2Vec trains separate center and context matrices. Implementations may publish the center vectors, the context vectors, or a combination. This choice can change evaluation results. “The Word2Vec vector” is therefore shorthand for a representation extracted from the trained parameters.

6.3 CBOW in one picture

Continuous Bag of Words reverses the task:

```
Skip-gram:  student → curious, reads, the, ...
CBOW:       the + curious + reads + the → student
```

CBOW combines nearby context vectors—often by averaging—and predicts the center word. It is usually faster; Skip-gram often represents infrequent words better. The deeper lesson is shared: **prediction forces the embedding table to organize itself around distributional evidence**.

7 GloVe: learn from global counts

Word2Vec processes local windows as training examples. GloVe—**Global Vectors**—starts from the aggregated co-occurrence matrix (Pennington et al. 2014). It asks which properties of global counts should determine meaning.

7.1 Co-occurrence ratios carry information

Imagine comparing the contexts *ice* and *steam*:

Table 4: The ratio distinguishes contexts associated with one word from contexts shared by both or related to neither.

Probe word k	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	Ratio
`solid`	high	low	large
`gas`	low	high	small
`water`	high	high	near 1
`fashion`	low	low	near 1

The contrast between two co-occurrence profiles can reveal semantic relationships more clearly than either raw count alone.

7.2 The GloVe objective

Let X_{ij} be the number of times word j occurs in the context of word i. GloVe learns word vectors $\mathbf{w}_i$, context vectors $\widetilde{\mathbf{w}}_j$, and biases by minimizing

$$J = \sum_{i,j} f(X_{ij}) \left(\mathbf{w}_i^{\mathsf{T}} \widetilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 . \tag{4}$$

Read the equation in pieces:

1. X_{ij} records global co-occurrence.

2. The vector dot product plus biases predicts $\log X_{ij}$.
3. The squared error penalizes poor predictions.
4. The weighting function f prevents extremely rare and extremely frequent pairs from dominating.

The logarithm compresses the enormous range of corpus counts. The weighting function trusts informative counts while limiting the influence of very frequent pairs.

7.3 Word2Vec and GloVe compared

Table 5: Word2Vec and GloVe use different objectives but exploit the same distributional evidence.

Property	Word2Vec Skip-gram	GloVe
Immediate training signal	Local center–context pairs	Aggregated co-occurrence counts
Typical objective	Distinguish observed from sampled pairs	Reconstruct log co-occurrence
Uses global corpus statistics	Indirectly, through repeated sampling	Explicitly
Produces	Static dense word vectors	Static dense word vectors
Main intuition	Predict the company a word keeps	Compress global patterns of company

! Do not reduce the comparison to “neural versus count-based.” Both methods learn parameters with optimization, and both are closely connected to transformed co-occurrence statistics. Their difference lies mainly in how the corpus evidence enters the objective.

8 Using pretrained embeddings

Training high-quality embeddings requires a large, representative corpus and substantial computation. In practice, we often begin with published vectors.

8.1 Well-known static embeddings

Table 6: Three influential families of pretrained static word embeddings.

Family	Original idea	Common published example	Useful property
Word2Vec	Predict local contexts	Google News, 300 dimensions	Influential predictive embeddings

Family	Original idea	Common published example	Useful property
GloVe	Fit global co-occurrence counts	Wikipedia + Gigaword, 50–300 dimensions	Several convenient model sizes
fastText	Represent words through character n-grams	Common Crawl, 300 dimensions	Better handling of morphology and unseen forms

fastText extends the basic static-word idea by composing a word representation from character n-gram vectors (Bojanowski et al. 2017). Words such as *teach*, *teacher*, and *teaching* share subword pieces. This is especially valuable in morphologically rich languages and for unfamiliar forms.

8.2 Query GloVe with Gensim

```
import gensim.downloader as api

vectors = api.load("glove-wiki-gigaword-100")

print(vectors.similarity("coffee", "tea"))
print(vectors.similarity("coffee", "airplane"))
print(vectors.most_similar("language", topn=5))

analogy = vectors.most_similar(
    positive=["king", "woman"],
    negative=["man"],
    topn=5,
)
print(analogy)
```

The first download can take time; later calls use Gensim's local cache. Exact neighbors vary across models.

8.3 Use vectors through spaCy

spaCy is not an embedding algorithm. It is an NLP library whose medium and large pipelines include pretrained static vectors. This makes it useful for applied exploration.

```
python -m spacy download en_core_web_md
```

```
import spacy

nlp = spacy.load("en_core_web_md")
```

```
coffee = nlp.vocab["coffee"]
tea = nlp.vocab["tea"]
airplane = nlp.vocab["airplane"]

print(coffee.has_vector, coffee.vector.shape)
print(coffee.similarity(tea))
print(coffee.similarity(airplane))
```

⚠ The small English pipeline, `en_core_web_sm`, does not include a full table of static word vectors. Use `en_core_web_md` or `en_core_web_lg` for this exercise.

8.4 Inspect neighborhoods, not isolated scores

A similarity value becomes meaningful only through comparison. Ask several questions:

- Are obvious related words near one another?
- Are the neighbors synonyms, topical associates, or grammatical alternatives?
- Does the behavior change for rare words or names?
- Does the model work equally well for the languages and domains you need?

```
for query in ["doctor", "computer", "beautiful", "running"]:
    print(query, vectors.most_similar(query, topn=5))
```

8.5 Visualize a small selection carefully

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

words = [
    "king", "queen", "prince", "princess",
    "coffee", "tea", "juice", "water",
    "car", "bus", "train", "bicycle",
]

points = PCA(n_components=2).fit_transform(vectors[words])

fig, ax = plt.subplots(figsize=(9, 6))
ax.scatter(points[:, 0], points[:, 1], color="#0f6f78")

for word, (x, y) in zip(words, points):
    ax.annotate(word, (x, y), xytext=(5, 4), textcoords="offset
↪   points")

ax.set_title("PCA projection of selected GloVe vectors")
ax.set_xlabel("Principal component 1")
ax.set_ylabel("Principal component 2")
```

```
plt.tight_layout()
plt.show()
```

PCA creates a lossy two-dimensional projection. Similarities should be calculated in the original 100-dimensional space, not inferred only from the picture.

9 What static embeddings cannot do

The success of static embeddings should not hide their constraints.

9.1 One word, one vector

A static vocabulary stores one vector for `bank`, whether the sentence refers to money or a river:

```
She deposited money at the bank.
They sat on the river bank.
```

The learned vector blends evidence from both senses. A contextual model instead produces a representation conditioned on the surrounding sentence.

9.2 Corpus boundaries become model boundaries

An embedding can only reflect evidence available in its training data. Specialized terminology, recent events, dialects, and underrepresented languages may be poorly represented. Similarity is learned from a corpus, not discovered as an objective fact.

9.3 Frequency and vocabulary remain important

Rare words receive fewer reliable updates. Word-level models cannot construct a vector for an unseen spelling. fastText reduces this problem with character n-grams, but it does not eliminate missing knowledge.

9.4 Bias is part of the geometry

Corpora contain historical and social stereotypes. Distributional learning can encode those associations and reproduce them in nearest neighbors and analogies (Bolukbasi et al. 2016). The same geometric regularities that make embeddings useful can make them harmful.

> **!** An embedding measures patterns of usage in a corpus. It does not provide a neutral dictionary of human meaning.

9.5 Intrinsic success is not task success

Word similarities and analogies are **intrinsic evaluations**: they test the representation directly. The more important question is often **extrinsic**: does the representation improve a real classifier, search system, or other downstream task?

10 Putting the ideas together

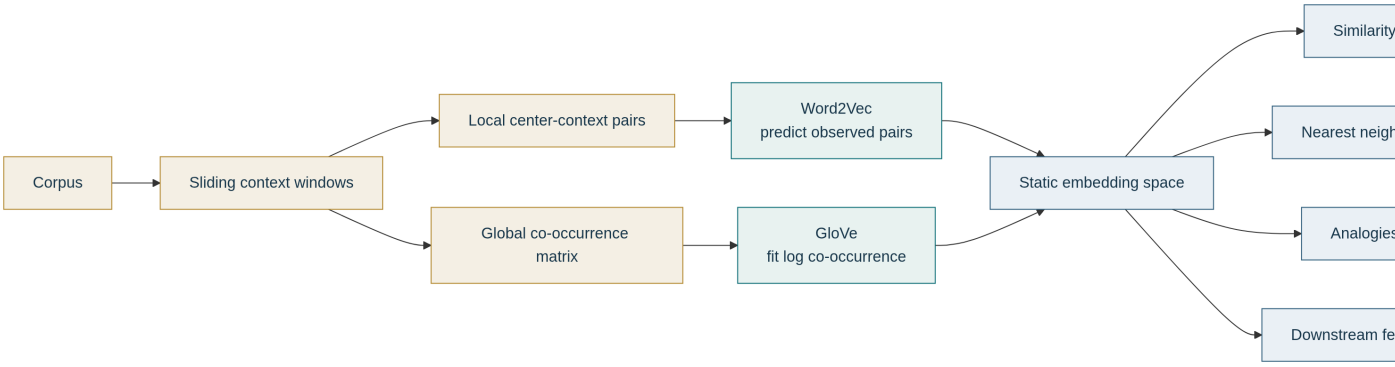


Figure 5: Two influential objectives turn the same distributional evidence into dense static vectors.

The conceptual chain is:

1. Token IDs identify vocabulary entries but have no semantic geometry.
2. One-hot vectors keep entries distinct but make every pair equally dissimilar.
3. Nearby contexts provide a scalable signal about word usage.
4. A co-occurrence matrix makes that signal explicit.
5. Word2Vec learns by predicting local context relationships.
6. GloVe learns by fitting global co-occurrence statistics.
7. Both produce static vectors that support similarity and analogy queries.
8. Static vectors remain corpus-dependent, biased, and unable to change with sentence context.

11 Check your understanding

1. Why is subtracting token IDs meaningless?
2. Why is the cosine similarity between two different one-hot vectors always zero?
3. With window radius two, which pairs are created around the center word in `students learn useful representations`?
4. What distinguishes a positive pair from a negative pair in Skip-gram training?
5. Why is negative sampling cheaper than computing a full softmax?
6. Why do words with similar contexts receive similar vectors even if they never occur beside each other?
7. In one sentence, contrast the training signals used by Word2Vec and GloVe.
8. Why might fastText handle an unfamiliar inflected word better than ordinary Word2Vec?
9. Why can `hot` and `cold` have high similarity despite being opposites?

10. Why should an analogy result not be treated as proof of semantic understanding?

Short answers

1. IDs are arbitrary vocabulary addresses; their arithmetic depends only on the chosen ordering.
2. Different one-hot vectors have no coordinate containing a nonzero value in common.
3. Generate all valid center–context pairs within two positions; edge words have fewer contexts.
4. A positive pair was observed in a corpus window; a negative pair was sampled as noise.
5. It scores only the positive context and a small number of negatives instead of every vocabulary item.
6. They repeatedly receive updates involving many of the same context vectors.
7. Word2Vec learns from local prediction pairs; GloVe explicitly fits aggregated co-occurrence counts.
8. fastText constructs words from character n-gram vectors, which unfamiliar forms may share with known words.
9. Antonyms frequently appear in similar syntactic and topical environments.
10. The result is a corpus- and model-dependent geometric regularity, not evidence of human-like comprehension.

12 Further reading and tools

12.1 Primary and explanatory readings

- Mikolov et al., *Efficient Estimation of Word Representations in Vector Space* (Mikolov, Chen, et al. 2013).
- Mikolov et al., *Distributed Representations of Words and Phrases and their Compositionality* (Mikolov, Sutskever, et al. 2013).
- Pennington, Socher, and Manning, *GloVe: Global Vectors for Word Representation* (Pennington et al. 2014).
- Bojanowski et al., *Enriching Word Vectors with Subword Information* (Bojanowski et al. 2017).
- Jurafsky and Martin, *Speech and Language Processing* (Jurafsky and Martin 2026).
- Hewitt, *CS224N Lecture 1 Notes* (Hewitt 2023).

12.2 Practical tools

- Gensim downloader — discover and load published embedding models.
- Gensim `KeyedVectors` — similarities, neighbors, and analogies.
- GloVe project — original code and pretrained vectors.
- fastText pretrained vectors — vectors for many languages.
- spaCy models — practical NLP pipelines, including packages with static vectors.

12.3 References

Bojanowski, Piotr, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2017. "Enriching Word Vectors with Subword Information." *Transactions of the Association for Computational Linguistics* 5: 135–46. https://doi.org/10.1162/tacl_a_00051.

Bolukbasi, Tolga, Kai-Wei Chang, James Zou, Venkatesh Saligrama, and Adam Kalai. 2016. "Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings." *Advances in Neural Information Processing Systems* 29. https://papers.nips.cc/paper/6228-man-is-to-computer-programmer-as-woman-is-to-homemaker-debiasing-word-embeddings.

Firth, John Rupert. 1957. "A Synopsis of Linguistic Theory, 1930–1955." In *Studies in Linguistic Analysis*. Blackwell.

Hewitt, John. 2023. *CS224N Lecture 1 Notes: Introduction and Word2Vec*. Stanford University. https://web.stanford.edu/class/cs224n/readings/cs224n_winter2023_lecture1_notes_draft.pdf.

Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd ed. https://web.stanford.edu/~jurafsky/slp3/.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Efficient Estimation of Word Representations in Vector Space." *arXiv Preprint arXiv:1301.3781*, ahead of print. https://doi.org/10.48550/arXiv.1301.3781.

Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. 2013. "Distributed Representations of Words and Phrases and Their Compositionality." *Advances in Neural Information Processing Systems* 26. https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. 2014. "GloVe: Global Vectors for Word Representation." *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1532–43. https://doi.org/10.3115/v1/D14-1162.